

MC521 - Relatório I

Júlio da Silva Telles

12 de Julho de 2026

1 F - P06 (AtCoder - abc251_d)

O problema F descreve uma situação em que dado um inteiro w tal que $1 \leq w \leq 10^6$ e necessário encontrar uma família de elementos descrita como $W = \{w_i \in \mathbb{Z}\}_{i \in I}$ com $1 \leq |I| \leq 300$, tal que $\forall n \in \mathbb{Z} \cap (1, w)$, $\exists A \subseteq I$ t.q. $1 \leq |A| \leq 3$ e $n = \sum_{a \in A} w_a$.

1.1 Solução

Como w não pode ser maior que 10^6 e não existe limitações adicionais ao problema podemos apenas formar uma solução para 10^6 que será viável para todos os possíveis valores de w . Tendo isso em mente dividimos a família em três secções de 100 elementos cada, descritos como:

$$W_1 = \{i \cdot 100^0 \mid i \in \mathbb{Z}, 1 \leq i \leq 100\}$$

$$W_2 = \{i \cdot 100^1 \mid i \in \mathbb{Z}, 1 \leq i \leq 100\}$$

$$W_3 = \{i \cdot 100^2 \mid i \in \mathbb{Z}, 1 \leq i \leq 100\}$$

Assim para qualquer inteiro $1 \leq w \leq 10^6$ temos 3 casos:

- se $1 \leq w \leq 10^2$, para todo w podemos formá-lo utilizando apenas um elemento do conjunto W_1 .
- se $10^2 < w \leq 10^4$, podemos formar w somando exatamente dois elementos: um pertencente a W_1 representando as duas primeiras casas decimais e outro a representando as últimas duas W_2 .
- se $10^4 < w \leq 10^6$, podemos formar w somando exatamente três elementos distintos: um de W_1 , um de W_2 e um de W_3 com o terceiro representando as duas últimas casas decimais expandindo do último caso.

Desse modo, é necessário apenas que para todo w a resposta seja a união entre W_1 , W_2 e W_3 .

2 G - P07 (AtCoder - abc249_d)

O problema nos dá uma sequência de inteiros $A = (A_1, \dots, A_N)$ de tamanho N , e pede como resposta o número de triplas de índices (i, j, k) que satisfaçam as seguintes condições:

- $1 \leq i, j, k \leq N$

- $\frac{A_i}{A_j} = A_k$
- $1 \leq N \leq 2 \times 10^5$
- $1 \leq A_i \leq 2 \times 10^5$ ($1 \leq i \leq N$)

2.1 Solução

Vamos resolver esse problema usando combinatoria, para isto primeiro precisamos reorganizar todos os A_i da sequência em um *Heat-map* de tamanho $K = 2 \times 10^5 + 1$ assim podemos guardar o numero de ocorrencias de cada numero da sequência e teremos uma variavel *max* que guarda qual é o maior numero da sequência, ademais devemos manipular a expressão $\frac{A_i}{A_j} = A_k$ para $A_i = A_j \times A_k$ para facilitar o trabalho de encontrar as triplas. Tendo esse *Heat-map* que, relaciona o indice do Array com o numero de ocorrencias daquele índice na sequência, podemos encontrar para todo inteiro i nessa sequência o numero de triplas de indices que o envolvem e satisfazem as condições estabelecidas, utilizando um *for* sobre todos os indices a partir do i escolhido até que $i \times j$ seja maior que $max + 1$, multiplicando as ocorrencias dos indices i, j e $i \times j$ (que nessa construção são os numeros dos indices correspondentes a j, k, i respectivamente)

```

1 for (int i = 1; i < max + 1; i++){
2     for (int j = i; j * i < max + 1; j++){
3         /* 0 if abaixo verifica se os indices sao distintos
4            e multiplicando a resposta por 2, contando a possivel
5            permutacao de indices na multiplicacao da tripla */
6         if (i != j) res += (o[i] * o[j] * o[i * j]) * 2;
7         else res += o[i] * o[j] * o[i * j];
8     }
9 }

```

2.2 Análise de Complexidade

Seja $N = max$. O laço externo itera sobre a variável i de 1 até N . Para cada valor de i , o laço interno itera sobre a variável j começando de i enquanto a condição $i \times j \leq N$ for verdadeira. Isso implica que, para um dado i , o laço interno executa no máximo $\frac{N}{i}$ vezes.

Para determinar o número total de iterações, devemos somar as execuções do laço interno para todos os possíveis valores do laço externo:

$$\sum_{i=1}^N \frac{N}{i} = N \sum_{i=1}^N \frac{1}{i}$$

Sabemos que o somatório $\sum_{i=1}^N \frac{1}{i}$ representa a série harmônica, cuja soma diverge logaritmicamente. Podemos aproximá-la da seguinte forma:

$$\sum_{i=1}^N \frac{1}{i} \approx \log N$$

Substituindo essa aproximação na nossa equação original, concluímos que o número de operações é proporcional a:

$$N \log N$$

Portanto, a complexidade de tempo do algoritmo é limitada por $\mathcal{O}(N \log N)$, tornando a execução viável para a densidade de dados apresentados pelo problema.